Internal Assessment Task II - Agentic AI System using Modern LLM-based Tools

Atlas: An Autonomous Travel Planning Agent

by

Roll No	Name
16015024036	Manthan Kadu
16015024040	Aditya Lokhande
16015024041	Aryan Lomte

For the subject

Artificial Intelligence

Department of Electronics & Computer Engineering

Academic Year 2026-27

Atlas: An Autonomous Travel Planning Agent

Group of 3: only to be made in same lab batch. All three members are from the same lab batch and contributed equally across the entire system.

Demo: will be taken in lab duration. The agent runs live from the browser console, and `python demo_agent.py` runs the same loop in the terminal with no browser required.

Live deployment: https://atlas-travel-agent-theta.vercel.app

Repository: https://github.com/Aryan-lomte05/Atlas.git

Introduction, Abstract of problem statement

Planning a multi-day trip is a research task before it is a writing task. A traveller must establish what the weather will actually be like in the chosen month, what a night costs at the destination, how to get there, which sights are worth the time, where to eat, whether a visa is required, and whether the total fits the budget. Each of those answers lives in a different place, and several of them change the others: a forecast of nine wet days changes which days the outdoor sights go on, and a lodging rate that overshoots changes the tier of the whole plan.

A conventional chatbot fails at this in a specific and predictable way. It produces a fluent, well-structured itinerary containing invented hotel rates, invented flight prices and invented weather, because it is generating the most likely continuation of a travel plan rather than consulting anything. The output is confident and unusable. The failure is not that the model is weak; it is that generation is the wrong operation for a task whose answer depends on facts the model does not hold.

Atlas is our response: an autonomous agent that treats trip planning as a loop of investigation rather than an act of composition. Given one goal in plain language, Atlas decides what it needs to know, calls a real tool to find out, reads the result, and uses it to decide the next step. It repeats this until the goal is satisfied, then writes an itinerary in which every figure is attributable to a tool call made during that run.

Three properties define the system:

1. **Grounded.** A figure may only enter the plan through a tool observation. Arithmetic is executed by a calculator, never by the model.
2. **Observable.** Every thought, action and observation is streamed to the interface as it happens and persisted afterwards, so the agent's behaviour can be audited step by step rather than taken on trust.
3. **Specific.** A plan must name real places. "Dinner at Britannia and Co." is acceptable output; "try a local Parsi restaurant" is not, because the traveller cannot act on it without doing the research themselves.

The project also implements a classical rule-based travel agent of the kind covered in the syllabus, and runs both against the same prompts from the same console, so the comparison in Part C is observed rather than asserted.

Scale of the delivered system: 3,267 lines of TypeScript and Python across the agent orchestrator, 16 tools, the classical agent, the streaming API and the interface.

AI framework(s) used

Layer	Choice	Why this choice
Agent pattern	**ReAct** (Reason + Act), implemented directly	The brief lists ReAct-style agents as an accepted framework. We wrote the loop ourselves rather than importing LangChain or CrewAI, so the control flow is readable in roughly 200 lines and every decision can be explained at the viva. A framework would have hidden exactly the mechanism being assessed.
Reasoning model	**openai/gpt-oss-120b** on **Groq**, resolved at run time	Free tier, 131,072 token context, fast enough that a full run finishes while the examiner watches.
Model access	Groq OpenAI-compatible API via the official `groq-sdk`	Free, no credit card, very low latency on their LPU inference hardware.
Runtime	Next.js 16 (App Router), React 19, TypeScript 5	One codebase serves the interface and the agent API route, and the same orchestrator is importable by a standalone Express server for split deployment.
Streaming	Server-Sent Events	Makes the trace appear step by step in the browser, which is what makes behaviour observable rather than a log dumped at the end.
Persistence	MongoDB Atlas via Mongoose	Stores completed traces so runs remain reviewable after the process exits.
Arithmetic	mathjs	Deterministic expression evaluation, deliberately outside the model.
Interface	Tailwind CSS v4, Lenis	A conversational console styled as a desktop chat window.

A note on the model, because it matters for reproducibility

Our original design named **Meta Llama 3.3 70B**. When we tested against the Groq catalogue with our API key, `llama-3.3-70b-versatile` was **not** in the list of models the account can serve. Every request returned an error and the agent silently fell back to its offline planner, which looked like success because a plan still appeared.

We fixed this properly rather than by swapping one hardcoded string for another. `selectModels()` queries the `/models` endpoint at run time, intersects the response with an ordered preference list, and returns every model the key can actually reach, best first:

```
const MODEL_PREFERENCE = [
  "llama-3.3-70b-versatile",
  "openai/gpt-oss-120b",
  "qwen/qwen3.8-27b",
  "openai/gpt-oss-20b",
  "llama-3.1-8b-instant",
];

async function selectModels(client: Groq): Promise<string[]> {
  if (resolvedModels) return resolvedModels;
  try {
    const list = await client.models.list();
    const available = new Set((list.data ?? []).map((m) => m.id));
    const usable = MODEL_PREFERENCE.filter((m) => available.has(m));
    resolvedModels = usable.length > 0 ? usable : [MODEL_PREFERENCE[1]];
  } catch {
    resolvedModels = [MODEL_PREFERENCE[1]];
  }
  return resolvedModels;
}
```

On our key this resolves to `openai/gpt-oss-120b`. If a future key can serve Llama 3.3 70B, the same code picks it first with no edit. Returning the whole list rather than a single id also lets a run drop to a smaller model mid-way

when the larger one is rate limited, which on a free tier is the difference between a finished demo and a fallback plan.

Tools integrated with agent

Atlas has **16 tools**. Eleven make live third-party calls, two compute locally, and three answer from maintained tables in the repository.

Every tool is labelled with which of the three it is, both here and in the interface. This matters: claiming that a curated table is a live API would be the easiest way to make the project look stronger than it is, and the distinction is exactly what an examiner should be able to check.

#	Tool	Source	Provider	Purpose
1	`geocode_place`	live	Open-Meteo geocoding	Resolve a name to coordinates, country, timezone, population
2	`weather_forecast`	live	Open-Meteo forecast	Seven day highs, lows, precipitation probability
3	`climate_normals`	live	Open-Meteo archive	What a given month is actually like, from the historical record
4	`daylight_hours`	live	sunrise-sunset.org	Sunrise, sunset and usable daylight for a date
5	`currency_convert`	live	Frankfurter (ECB rates)	Convert between currencies at today's published rate
6	`public_holidays`	live	Nager.Date	Closures and crowding that would wreck a day plan
7	`air_quality`	live	Open-Meteo air quality	European AQI and PM2.5
8	`elevation`	live	Open-Meteo elevation	Altitude, to warn about acclimatisation
9	`attractions_explorer`	live	Wikipedia REST	Encyclopedic description of a place or landmark
10	`find_attractions`	live	Wikivoyage, OSM fallback	Named sights a traveller has heard of
11	`find_restaurants`	live	Wikivoyage, OSM fallback	Named restaurants, cafes and bars
12	`budget_calculator`	compute	mathjs	Exact arithmetic, kept out of the model
13	`packing_list`	compute	local rules	Derived from observed temperature and rainfall
14	`route_options`	curated	in-repo table	Carriers, duration, indicative return fare
15	`lodging_tiers`	curated	in-repo table	Nightly rates across three tiers
16	`travel_advisory`	curated	in-repo table	Visa posture, safety, payment habits, etiquette

The brief requires a minimum of two tools. Atlas exceeds that by a factor of eight, and more importantly the tools are heterogeneous: geocoding, time series, financial data, civic calendars, encyclopedic text, curated travel guides, map data and local computation.

Part A - Agent Design (Modern View)

Goal of the agent

Take a single natural-language travel goal, for example:

> *"Plan 5 days in Kyoto in early April for two people on a 90,000 INR budget. Work around the weather."*

and return a day-by-day itinerary with an itemised budget, in which every factual claim originates from a tool call made during that run.

The goal is deliberately underspecified by design. Real requests omit the origin city, the travel style, and sometimes the budget. Part of the agent's job is to notice what is missing, choose a defensible default, and state that it has done so, rather than refusing or silently assuming.

The agent is also expected to let observations change the plan. If the climate record reports nine wet days, the outdoor days must move. If the first budget pass overshoots, it must re-price rather than quietly round down.

Agent role

Atlas is a **travel concierge**. The role is encoded in the system prompt (reproduced in full under "Prompts and output screenshots") and constrains behaviour along three axes:

Grounded. It may not state a temperature, rate, fare or total that did not come from an observation in the current run. Where no tool can supply a figure, it says so in the plan instead of producing a plausible one.

Specific. The plan must name real places, drawn from the place tools, and not fall back to generic advice. A plan the traveller cannot act on without further research has not done its job.

Honest about provenance. Figures from curated tables are described as indicative; figures from live APIs are reported as measured.

Tools used

The sixteen tools are listed above and documented in detail in Part B. The governing design principle is:

> ***The model supplies language and judgement. The tools supply facts.***

Arithmetic in particular is never left to the model, because invented arithmetic is the most common and least visible failure mode in LLM-generated plans: a budget table that sums incorrectly looks exactly like one that does not.

Memory type (short-term / context)

Atlas uses three distinct kinds of memory.

Short-term, within a run: the scratchpad

An ordered transcript of every thought, action and observation so far, re-read in full before every decision. This is what makes the loop stateful: step 7 can only be chosen because steps 1 to 6 are visible to the model.

```
Goal: Plan 5 days in Kyoto in early April for two people on 90000 INR

Thought: I need coordinates before any location tool.
Action: geocode_place
Action Input: Kyoto
Observation: Kyoto, Japan (JP) / lat 35.0, lon 135.8 / Asia/Tokyo

Thought: April is months away, a forecast will not reach that far.
Action: climate_normals
Action Input: Kyoto, 4
Observation: mean high 19.3 C, mean low 8.4 C, wet days 9 of 30
```

When the scratchpad approaches the context limit it is trimmed from the middle, preserving the goal framing at the head and the most recent exchanges at the tail:

```
function trimScratchpad(pad: string): string {
  if (pad.length <= MAX_SCRATCHPAD_CHARS) return pad;
  const head = pad.slice(0, 1500);
  const tail = pad.slice(-(MAX_SCRATCHPAD_CHARS - 1600));
  return `${head}\n\n[...earlier steps trimmed to fit the context window...]\n\n${tail}`;
}
```

Contextual, across turns in a conversation

The console keeps the thread. A follow-up such as *"make it cheaper"* or *"what should I pack?"* is sent with the previous goals and answers as settled context, so the agent amends the existing plan instead of re-running every tool from the first request.

```
let scratchpad = priorContext
  ? `Earlier in this conversation:\n${priorContext}\n\nFollow-up goal: ${goal}\n`
  : `Goal: ${goal}\n`;
```

Long-term, across runs: the episodic store

Each completed run is written to MongoDB Atlas (collection `agenttraces`) with its full step list, tool payloads, timings and final answer, so past runs remain queryable after the process exits.

Memory	Scope	Lifetime	Medium
Scratchpad	One run	Duration of the run	In-process string
Contextual	One session	Until the page is reloaded	Client state, sent as context
Episodic	All runs	Permanent	MongoDB Atlas

Planning strategy (ReAct / step-by-step)

ReAct, chosen over plan-then-execute precisely because the observations change the plan. The agent does not decide all its steps in advance. Each iteration is:

```
Thought       what I still need to establish, and why this tool settles it
Action        one tool name
Action Input  the argument
Observation   the tool result, appended verbatim
```

repeated until the goal is met or a ceiling of 14 iterations is reached.

Why not plan-then-execute? Because a fixed plan written before step 1 cannot know that the climate record will report nine wet days, and therefore cannot contain the step that reorders the itinerary around them. The adaptivity is the point.

Prompting alone did not produce reliable behaviour. Four guarantees are enforced in orchestrator code rather than requested in the prompt. They are the most substantial engineering in the project and are covered under "Multi-step reasoning" in Part B.

Part B - Implementation (Modern AI Tools)

LLM-based agent

Model: `openai/gpt-oss-120b` **served by Groq**, selected at run time from an ordered preference list against the models the key can actually reach.

Justification for the choice

Criterion	Assessment
Cost	Free tier. A paid model was not viable for a student project, and Groq's free tier made the whole thing possible.
Context window	131,072 tokens. The ReAct scratchpad grows with every observation; a 4k or 8k window would force aggressive truncation by step five or six, and the agent would begin forgetting facts it had just gathered.
Latency	Roughly one second per reasoning step on Groq's LPU hardware. A full run of 10 to 14 iterations completes in about 60 to 200 seconds, which is the difference between a live demo and a recorded video.
Instruction adherence	Holds the Thought / Action / Action Input format reliably enough to be parsed. Smaller models we tested drifted out of format within a few steps.
Open weights	`gpt-oss` is an open-weights model, so the system is not dependent on a single proprietary vendor.

Measured performance across our recorded runs

Metric	Observed range
Loop iterations per run	9 to 16
Distinct tools called per run	6 to 11
Total trace steps	29 to 47
Wall-clock duration	61 s to 240 s
Tool call success rate	100% (failures degrade to an informative observation rather than an exception)

Tools used: list, justification of choice, features

Each tool returns a plain string that is appended to the scratchpad verbatim. The strings are written to be read by a language model: short labelled lines, explicit units, and an interpretation line wherever a raw number would not be actionable on its own.

Live tools, with real output

1. `geocode_place(place name)` - Open-Meteo geocoding

```
Kyoto, Japan (JP)
lat 35, lon 135.8
timezone Asia/Tokyo, population 1,463,723
```

2. `weather_forecast(place name)` - Open-Meteo forecast

```
Mumbai, India, next 7 days
Sun 2026-10-11: 37.2/28.6 C, light drizzle, rain 31%
Mon 2026-10-12: 37.3/26.8 C, mainly clear, rain 0%
Wet days (50% or more): 0 of 7.
Mostly workable for outdoor days.
```

3. `climate_normals(place, month)` - Open-Meteo historical archive

```
Kyoto in April, measured from the 2025 record
mean high 19.3 C, mean low 8.4 C
wet days 9 of 30, total 82.4 mm
Mostly dry. Outdoor days are safe to fix in advance.
```

4. `daylight_hours(place, ISO date)` - sunrise-sunset.org

```
Reykjavik on 2026-11-15
sunrise 09:53, sunset 16:31 (Atlantic/Reykjavik)
daylight 6h 38m
Very short day. Plan one outdoor block and keep it early.
```

5. `currency_convert(amount from to)` - Frankfurter, ECB reference rates

```
500 USD = 48365.00 INR
rate 1 USD = 96.7300 INR
ECB reference rate, 2026-10-09
```

6. `public_holidays(country code, year)` - Nager.Date

```
JP public holidays 2026 (16 total, first 14 shown)
2026-04-29  Showa Day
Museums and offices often close on these dates, and domestic travel peaks.
```

7. `air_quality(place)` - Open-Meteo air quality

```
Delhi: European AQI 164 (extremely poor), PM2.5 78.4 ug/m3
Keep outdoor blocks short and prefer early morning.
```

8. `elevation(place)` - Open-Meteo elevation

```
Leh: 3524 m above sea level
Serious altitude. Allow two or three acclimatisation days, keep day one light.
```

9. `attractions_explorer(place or landmark)` - Wikipedia REST

```
Kinkaku-ji (Buddhist temple in Kyoto, Japan)
Kinkaku-ji, officially named Rokuon-ji, is a Zen Buddhist temple in Kyoto,
Japan and a tourist attraction...
```

10. `find_attractions(place)` - Wikivoyage, OpenStreetMap fallback

```
Places to see in Mumbai, from the Wikivoyage travel guide

WORTH SEEING
- Gateway of India: The landmark gateway was built in 1911 to commemorate
  the visit of King George V and Queen Mary to India.
- Sassoon Docks: One of the oldest docks in Mumbai and the first wet dock
  constructed in the city. Also one of the largest fish markets.
- Wellington Fountain: A small elegant marble fountain dating to the 1860s.
```

11. `find_restaurants(place)` - Wikivoyage, OpenStreetMap fallback

```
Where to eat and drink in Mumbai

EAT
- Bademiya Kebab Restaurant: Eat on the road or in your car. Open until
  the early hours.
- Martin's: Great Goan food.

DRINK
- Cafe Mondegar: Serves excellent pitchers of beer.
- Bootlegger's: A small popular pub off Causeway.
```

Local compute

12. `budget_calculator(expression)` - mathjs

```
31400 + 6200*5 + 9000 + 15200
= 86600
```

13. `packing_list(minC, maxC, rainPct, notes)` - local rules

```
Conditions: 8 to 19 C, rain 55%
- mid layers
- a light insulated jacket
- waterproof shell
- compact umbrella
- broken-in walking shoes
- shoulders and knees covered for religious sites
```

Curated tables

14. `route_options(destination)`, **15.** `lodging_tiers(destination)`, **16.** `travel_advisory(country or city)` - maintained tables covering 14 corridors, 11 destinations and 11 countries respectively. Each states in its output that the figures are indicative and from a maintained table, and when a destination is absent it says so and instructs the agent to flag the gap rather than invent a number.

Justification of three notable design decisions

Why `climate_normals` exists alongside a forecast. A forecast reaches seven days. Most trips are planned further out, so a forecast is simply the wrong instrument, and asking for next April's weather in October returns nothing useful. This tool queries the historical archive for the same calendar window in the previous year. It is what allows the agent to plan an April trip in October, which is the normal case.

Why the place tools changed source mid-project. Our first implementation used only the OpenStreetMap Overpass API. A Mumbai query returned:

```
- Kings Circle (garden)
- bialbazzar (park)
- NSm park (park)
- park (park)
- Domino's (restaurant, pizza)
- Cafe Coffee Day (cafe)
```

These are mapped, but nobody flies to Mumbai for them. OSM contains everything that has been mapped, not everything worth visiting. We switched the primary source to **Wikivoyage**, a human-written travel guide whose pages carry structured `see`, `do`, `eat` and `drink` listings. The same query now returns the Gateway of India, Sassoon Docks, Marine Drive, Bademiya and Cafe Mondegar. Overpass is retained as a fallback for places Wikivoyage does not cover.

Two implementation details were needed. Wikivoyage splits large cities across district subpages, so the tool merges them when the parent page is thin, and it triggers that merge on the count of `see` and `eat` entries

specifically rather than the total, because a hub page is often full of festival entries under `do` while every real listing sits on a subpage. Second, listings are wiki templates that can nest, so a brace-matching parser is used rather than a regular expression.

Why three tools are curated rather than live. Live flight-fare and hotel-booking APIs are paid, rate limited and require commercial agreements. Rather than pretend otherwise, these three answer from maintained tables and are labelled "curated" everywhere they appear. When a destination is absent they instruct the agent to flag the gap, which produced one of the better behaviours we observed: for a Mumbai run the agent wrote *"Specific flight or train fares to Mumbai are not available from curated data; the 4,000 INR allocated for transport is for local activities and intra-city travel only."*

Multi-step reasoning

A plain ReAct loop over a capable model behaves worse than the literature suggests. We observed three concrete failures in live testing and fixed each in code. This is the core engineering contribution of the project.

Failure 1: premature completion

The model gathered three facts, decided it had the gist, and wrote a confident itinerary whose lodging and transport figures it had invented:

```
tools: ['geocode_place', 'climate_normals', 'daylight_hours']
-> Final Answer with a full budget table
```

The budget was fiction.

> ***Mechanism 1: the grounding gate.*** *A `Final Answer` is rejected while any required tool remains uncalled. The agent receives an observation naming exactly which facts are still ungrounded and is sent back to the tools.*

```
function requiredTools(goal: string): string[] {
  const required = ["geocode_place", "budget_calculator"];
  required.push("lodging_tiers", "route_options");
  required.push("find_attractions", "find_restaurants");
  required.push(extractMonth(goal) ? "climate_normals" : "weather_forecast");
  if (/visa|passport|safe|entry|document/.test(goal.toLowerCase())) {
    required.push("travel_advisory");
  }
  return required;
}

if (fa) {
  const missing = missingTools(goal, toolsUsed);
  if (missing.length > 0 && nudges < MAX_NUDGES) {
    nudges++;
    const note =
      `Not ready for a Final Answer. These facts are still ungrounded because ` +
      `the tool was never called: ${missing.join(", ")}. Any figure you would ` +
      `write for them right now would be invented. Call ${missing[0]} next.`;
    emit({ type: "observation", content: note });
    scratchpad += `\n${reply}\nObservation: ${note}\n`;
    continue;
  }
  finalAnswer = fa;
  emit({ type: "answer", content: fa });
  break;
}
```

The required set is derived from the goal, so a goal mentioning visas also requires `travel_advisory`. This mechanism is visible firing in `docs/logs/kyoto-gate-and-429.log`.

Failure 2: breadth-first exploration

Given room, the agent queried sights repeatedly and priced the same corridor from three different origin cities, exhausting its iteration budget before pricing anything:

```
ACTION: route_options(Kyoto, Delhi)
ACTION: route_options(Kyoto, Mumbai)
ACTION: route_options(Kyoto, Bangalore)
ACTION: attractions_explorer(Kinkaku-ji, Kyoto)
ACTION: attractions_explorer(Kinkaku-ji)
```

> ***Mechanism 2: per-tool call caps and normalised deduplication.***

```
const TOOL_CALL_CAP: Record<string, number> = {
  geocode_place: 2,  route_options: 1,   lodging_tiers: 1,
  travel_advisory: 1, climate_normals: 1, public_holidays: 1,
  attractions_explorer: 3, find_attractions: 2, find_restaurants: 2,
  budget_calculator: 4,
};

// "Kinkaku-ji, Kyoto" and "Kinkaku-ji" are the same call for this purpose.
const callKey = `${tool}::${input.toLowerCase().replace(/[^a-z0-9]/g, "")}`;
```

> ***Mechanism 3: iteration budgeting.*** *When the remaining iterations approach the number of required tools still unrun, the orchestrator injects an observation naming them and instructing the agent to call them immediately.*

```
const stillMissing = missingTools(goal, toolsUsed);
const left = MAX_ITERATIONS - i - 1;
if (stillMissing.length > 0 && left <= stillMissing.length + 1) {
  const note =
    `${left} steps remain and ${stillMissing.length} required tools have ` +
    `not run: ${stillMissing.join(", ")}. Call them now, in that order, ` +
    `before writing anything else.`;
  emit({ type: "observation", content: note });
  scratchpad += `Observation: ${note}\n`;
}
```

Failure 3: the model ignoring instruction anyway

Even with all of the above, some runs reached the ceiling without computing a budget. The steering observation appeared in the trace and the model continued querying sights regardless.

> ***Mechanism 4: deterministic enforcement.*** *If the loop ends and `budget_calculator` was never called, the orchestrator derives the expression itself from figures already in the trace, executes it, appends the result as a real step, and regenerates the answer.*

```
function deriveBudgetExpression(goal: string, steps: AgentStep[]): string | null {
  const obsFor = (tool: string) =>
    steps.find((s) => s.type === "observation" && s.tool === tool)?.content ?? "";

  const fare = Number(
    obsFor("route_options").match(/indicative return:\s*([\d,]+)/)?.[1]
      ?.replace(/,/g, "") ?? 0,
  );
  const nightly = Number(obsFor("lodging_tiers").match(/mid-range\s+(\d+)/)?.[1] ?? 0);
  if (!fare && !nightly) return null;   // never invent the inputs either

  const nights = extractNights(goal);
  const travellers = extractTravellers(goal);
  const parts: string[] = [];
  if (fare) parts.push(`${fare}*${travellers}`);
  if (nightly) parts.push(`${nightly}*${nights}`);
  parts.push(`2800*${nights}*${travellers}`);
  return parts.join(" + ");
}
```

The consequence is the strongest guarantee in the system: a total in an Atlas plan is never a number the model wrote. It is either computed by mathjs at the model's request, or computed by mathjs at the orchestrator's insistence.

Two further robustness measures

Parser hardening. Our first parser was a regular expression with lookaheads. Models wrap labels in markdown (`**Action:**`), sometimes emit two cycles in one reply, and sometimes hallucinate their own `Observation`. The regex swallowed all of it into the tool argument, producing calls such as:

```
ACTION: geocode_place(** Kyoto**Thought:** I need the typical weather for...)
ACTION: geocode_place(MumbaiAction: find_attractions)
```

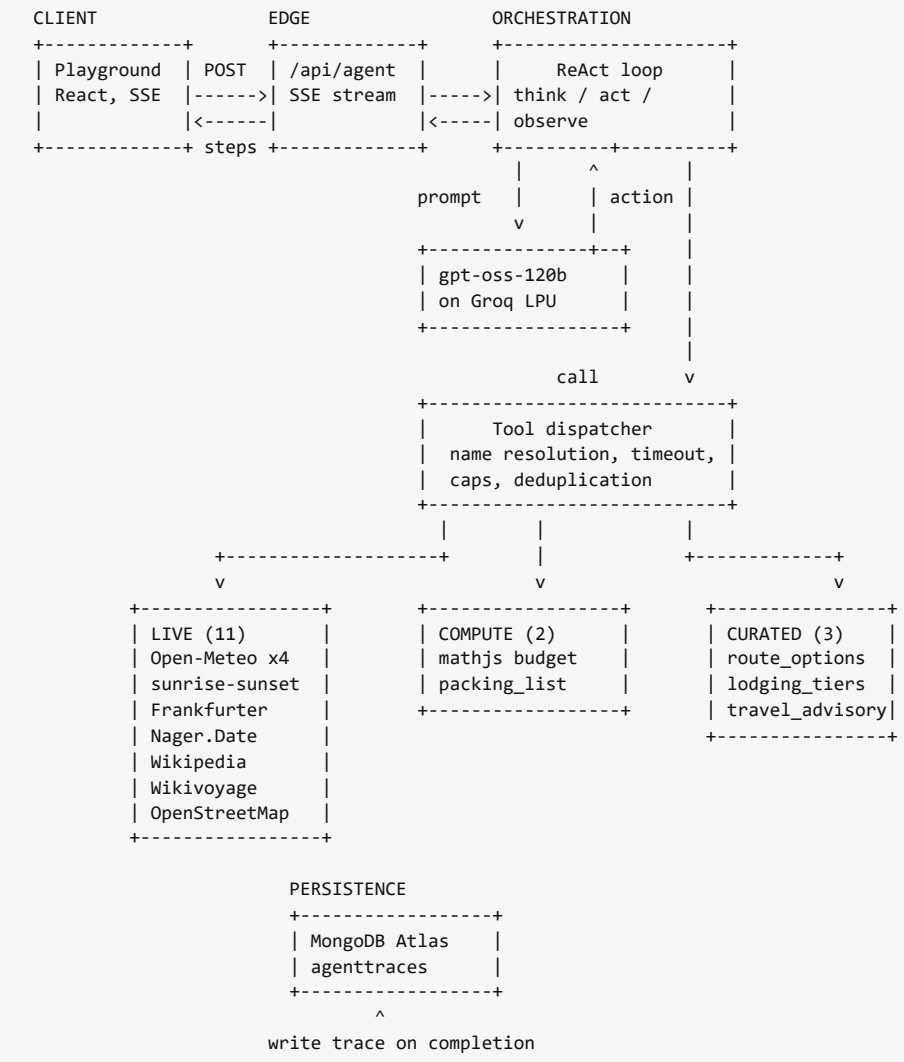
It was replaced with a line-based parser that strips markdown emphasis, accumulates each labelled block, honours only the first cycle in a reply, and ignores a `Final Answer` that appears after an `Action`, so a reply that reasons, acts, then speculates about the answer still executes the tool.

Rate-limit recovery. Groq's free tier enforces a token-per-minute cap. The first implementation treated a 429 as fatal and dropped to the offline planner, discarding every tool result already gathered and re-running all of them. Now the run first tries switching to a smaller available model, and failing that waits the interval stated in the error body:

```
function retryAfterMs(error: string, attempt: number): number {
  const stated = error.match(/try again in ([\d.]+)\s*s/i);
  if (stated) return Math.min(30000, Math.ceil(parseFloat(stated[1]) * 1000) + 400);
  return Math.min(20000, 1500 * 2 ** attempt);
}
```

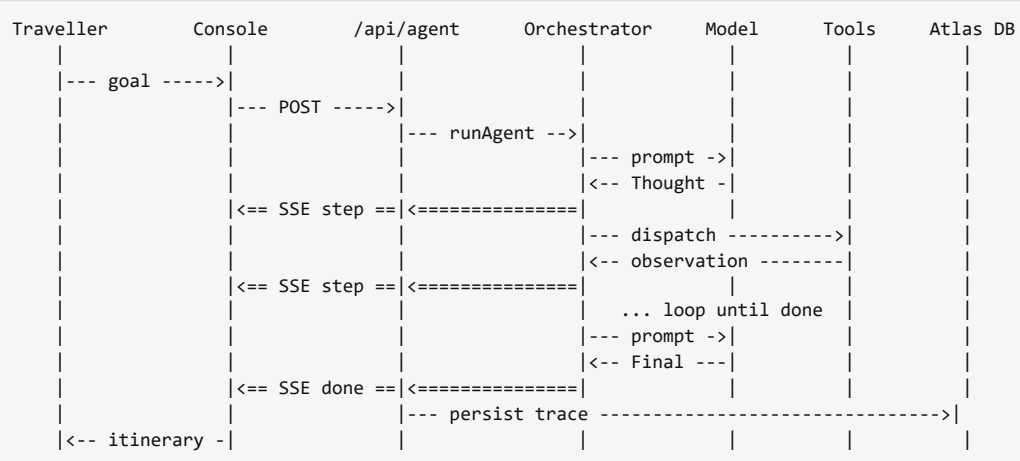
If the model becomes permanently unreachable, the run assembles a partial plan from the observations already collected rather than failing outright. Every one of these paths is visible in the trace, which is the point: the agent explains its own degradation.

Architecture diagram



A rendered, themed version of this diagram is in the interface under the Architecture section; see Figure 4.

Request sequence



Tool flow diagram

A representative run for *"Plan 5 days in Kyoto in early April for two people on a 90,000 INR budget"*:

```
 goal
  |
  v
[1] Thought  "I need coordinates before any location tool"
    Action   geocode_place("Kyoto")
    Observ.  Kyoto, Japan (JP), lat 35.0, lon 135.8, Asia/Tokyo
  |
  v
[2] Thought  "April is months out, a forecast will not reach"
    Action   climate_normals("Kyoto, 4")
    Observ.  mean high 19.3 C, low 8.4 C, wet days 9 of 30
  |                                  |
  |                                  +--> drives day ordering
  v
[3] Action   daylight_hours("Kyoto, 2026-04-05")
    Observ.  sunrise 05:37, sunset 18:22, 12h 45m, two blocks fit
  |
  v
[4] Action   public_holidays("JP, 2026")
    Observ.  Golden Week from Apr 29, heavy domestic travel
  |
  v
[5] Action   find_attractions("Kyoto")      -> named sights, Wikivoyage
[6] Action   find_restaurants("Kyoto")      -> named places to eat
[7] Action   lodging_tiers("Kyoto")         -> 2400 / 6200 / 14500 INR
[8] Action   route_options("Kyoto")         -> via Osaka KIX, 62000 INR
  |
  v
[9] Thought  "I have the inputs, total them with the calculator"
    Action   budget_calculator("62000*2 + 6200*5 + 2800*5*2")
    Observ.  = 183000            <-- over the 90000 ceiling
  |
  v
[10] Thought  "Over budget. Re-price at the backpacker tier."
     Action   budget_calculator("62000*2 + 2400*5 + 1200*5*2")
     Observ.  = 148000
   |
   v
     Final Answer: day-by-day plan, itemised budget, honest note that
     the international fare alone exceeds the stated ceiling
```

The branch at step 9 is the point of the whole exercise. The second budget call exists only because the first one returned a number over the ceiling. A rule-based agent cannot produce that step, because nobody wrote a rule for it.

Observable agent behavior (logs / traces)

Every step is emitted the moment it completes and streamed to the browser over Server-Sent Events, so the trace fills in live rather than arriving as one block at the end. Four step types are distinguished and colour-coded:

Type	Meaning	Colour in the console
`thought`	The agent's reasoning before acting	Pink
`action`	The tool name and argument chosen	Coral
`observation`	The tool result, verbatim	Teal
`answer`	The finished itinerary	White

Observations are appended **verbatim**. Nothing is paraphrased on the way in, so a later step can be audited against exactly what the API returned.

The console offers one-click copy and `.txt` download of the whole transcript, which is how the committed logs were produced. Completed runs are also written to MongoDB Atlas with the full step list, tool payloads, timings and final answer.

Code repository / notebook

Repository: https://github.com/Aryan-lomte05/Atlas.git

Live deployment: https://atlas-travel-agent-theta.vercel.app

The deployed instance runs the full ReAct loop against the live tools with a Groq key configured, so the agent can be demonstrated from any browser with no local setup.

Path	Lines	Responsibility
`src/lib/agent.ts`	840	ReAct orchestrator, four enforcement mechanisms, parser, model selection, deterministic planner
`src/lib/tools.ts`	1181	16 tool implementations, Overpass and Wikivoyage clients, dispatcher
`src/lib/toolManifest.ts`	239	Single source of truth for the tool surface, shared by agent and interface
`src/lib/classicalAgent.ts`	177	Part C rule-based agent
`src/app/api/agent/route.ts`	130	SSE streaming endpoint, classical handler, trace persistence
`src/components/Playground.tsx`	648	Conversational console with live trace
`demo_agent.py`	182	Terminal demo, no browser required
`src/lib/db.ts`, `src/lib/models/`	110	MongoDB connection and schemas
`server/index.ts`	-	Standalone Express server for split deployment

Run locally:

```
git clone https://github.com/Aryan-lomte05/Atlas.git
cd Atlas
npm install
cp .env.example .env.local     # add GROQ_API_KEY from console.groq.com (free)
npm run dev                    # http://localhost:3000
python demo_agent.py           # terminal demo for the viva
```

Validate:

```
npx tsc --noEmit               # typecheck, passes clean
npm run build                  # production build, passes clean
```

Deployment. The system supports two shapes from one codebase. Unified on Vercel, where the App Router serves both the interface and `/api/agent`. Or split, with the same orchestrator imported by `server/index.ts` and deployed to Render via the included `render.yaml`, with the frontend pointed at it through `NEXT_PUBLIC_API_URL`. Neither `GROQ_API_KEY` nor `MONGODB_URI` is strictly required: without the key the deterministic planner runs the same live tools in a fixed order, and without the URI traces simply are not persisted. A lab machine with no credentials can still demonstrate the system.

Agent interaction logs

Four captured logs are committed in `docs/logs/`:

File	Steps	Tools	What it demonstrates
`kyoto-react-full.log`	35	10	A complete run across ten distinct tools
`kyoto-steering.log`	40	6	Iteration budgeting injecting steering mid-run
`kyoto-gate-and-429.log`	47	9	The grounding gate firing, and rate-limit recovery
`mumbai-named-places.log`	9	1	Wikivoyage returning real named spots

An extract from `kyoto-react-full.log`:

```
[01] THOUGHT
     I need the basic location details for Kyoto to use in subsequent
     tools (weather, daylight, attractions, lodging).
[02] ACTION
     tool: geocode_place(Kyoto)
[03] OBSERVATION
     Kyoto, Japan (JP)
     lat 35, lon 135.8
     timezone Asia/Tokyo, population 1,463,723
[04] THOUGHT
     The trip is in April, months away, so the forecast will not reach.
     The seasonal record is the right instrument.
[05] ACTION
     tool: climate_normals(Kyoto, 4)
[06] OBSERVATION
     Kyoto in April, measured from the 2025 record
     mean high 19.3 C, mean low 8.4 C
     wet days 9 of 30, total 82.4 mm
     Mostly dry. Outdoor days are safe to fix in advance.
[07] THOUGHT
     I need the sunrise, sunset and usable daylight for early April to
     ensure the daily plan fits within daylight hours.
[08] ACTION
     tool: daylight_hours(Kyoto, 2026-04-05)
[09] OBSERVATION
     Kyoto on 2026-04-05
     sunrise 05:37, sunset 18:22 (Asia/Tokyo)
     daylight 12h 45m
     A full day of light. Two outdoor blocks fit comfortably.
```

The grounding gate firing, from `kyoto-gate-and-429.log`:

```
[18] OBSERVATION
     Not ready for a Final Answer. These facts are still ungrounded
     because the tool was never called: budget_calculator, lodging_tiers,
     route_options. Any figure you would write for them right now would
     be invented. Call budget_calculator next.
```

Rate-limit recovery, from the same log:

```
[26] OBSERVATION
     openai/gpt-oss-120b is rate limited. Switching to openai/gpt-oss-20b
     and continuing with the 6 tool results already gathered.
```

Prompts and output screenshots

The system prompt

This is supplied to the model on every iteration, and is the specification of the agent's role.

```
You are Atlas, an autonomous travel planning agent.

You are given one goal. You reach it by repeating a single cycle: reason about
what you still need, call exactly one tool, read what it returns, then decide
again. You do not plan every step up front, because each observation can change
what the next step should be.

AVAILABLE TOOLS
  (the 16 tools, each with its argument shape and a one-line summary,
   generated from toolManifest.ts so the prompt can never drift from
   what the dispatcher actually implements)

OUTPUT FORMAT
Emit exactly one cycle per reply, in this shape and nothing else:

Thought: what you still need to establish, and why this tool settles it
Action: one tool name from the list above
Action Input: the argument, matching that tool's expected shape

Wait for the Observation. Do not write it yourself, and do not emit more than
one Action per reply.

When you have enough to write the plan, reply instead with:

Thought: why the goal is now satisfied
Final Answer: the itinerary

RULES THAT MATTER
1. Never state a temperature, rate, fare or total that did not come from a tool
   observation in this run. If no tool can supply it, say so in the plan rather
   than producing a figure that merely reads well.
2. Resolve the destination with geocode_place before any location tool, unless
   the place is obviously unambiguous.
3. For a trip more than a week out, use climate_normals. A forecast only
   reaches seven days and will not cover the dates.
4. Every total must come from budget_calculator. Do not do arithmetic yourself.
5. Check daylight_hours for any destination far from the equator. Daylight, not
   opening hours, is what limits a winter day at high latitude.
6. Do not call the same tool with the same input twice. If an observation was
   not useful, change the argument or change the tool.
7. Name real places. Call find_attractions and find_restaurants and build the
   days out of what they return: "dinner at Bastian" and "sunset at Bandra
   Fort", never "try a local seafood place" or "explore the old town". A plan
   the traveller cannot act on without further research has not done its job.
8. Stop and write the Final Answer as soon as the goal is met. Extra calls that
   do not change the plan are wasted steps.

THE FINAL ANSWER
Write it for the traveller, not as a data dump. Give a day by day plan, then an
itemised budget table with a total and the headroom against any stated ceiling,
then a short list of what to know before going. Attribute each figure to the
tool that produced it. Where the weather drove a decision, say so explicitly.
```

Example user prompts used in testing

```
Plan 5 days in Kyoto in early April for two people on a 90000 INR budget.
  Work around the weather.
5 days in Mumbai for two. Tell me exactly where to eat and what to see.
A week in Reykjavik in November. No budget set. What should I know?
Four days in Goa for three friends, 18000 INR total.
Make it cheaper.                      (follow-up, uses conversational memory)
What should I pack?                   (follow-up)
```

Screenshots

All eleven are committed in `docs/screenshots/` and were captured from the running application against live APIs.

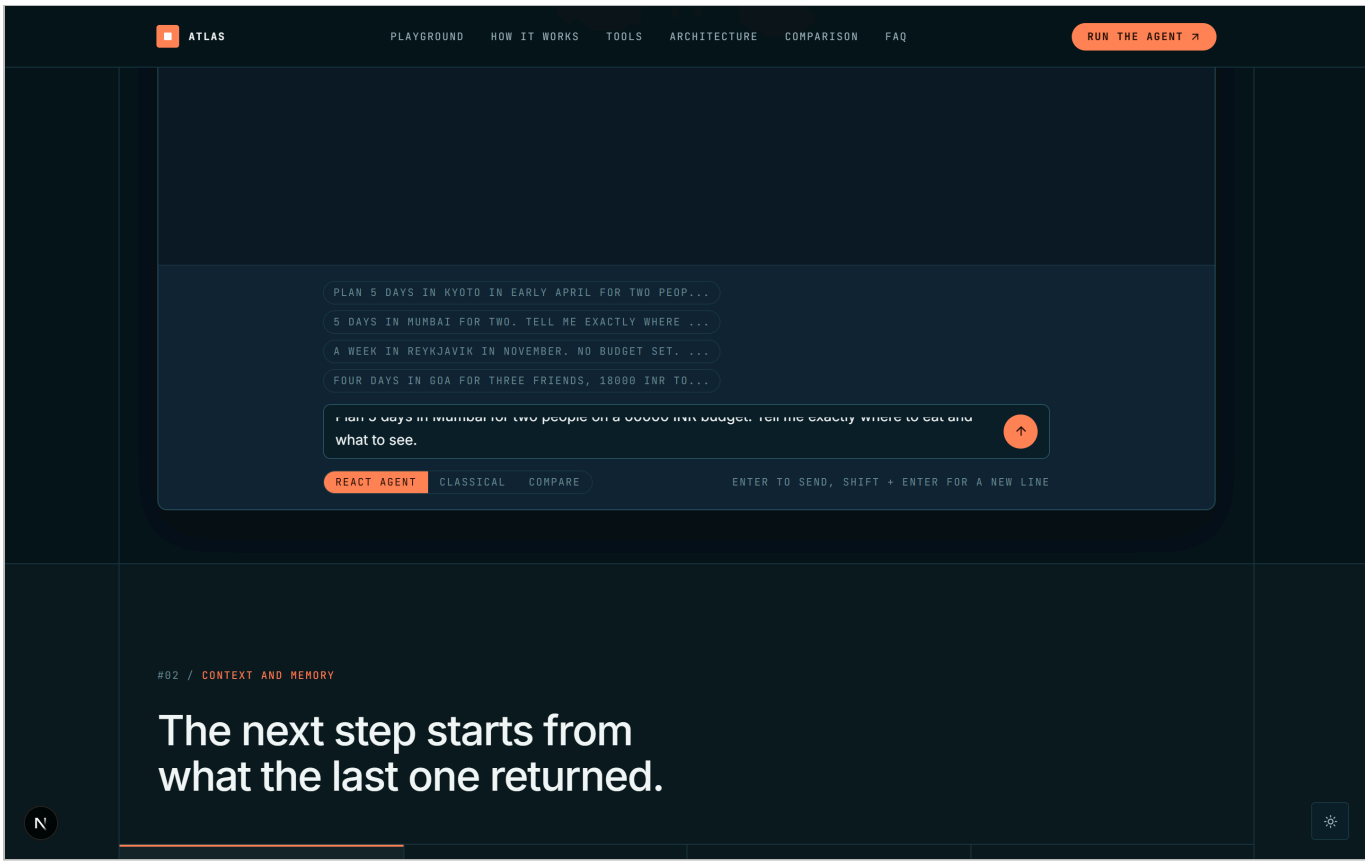


Figure 1: the prompt as entered in the console.

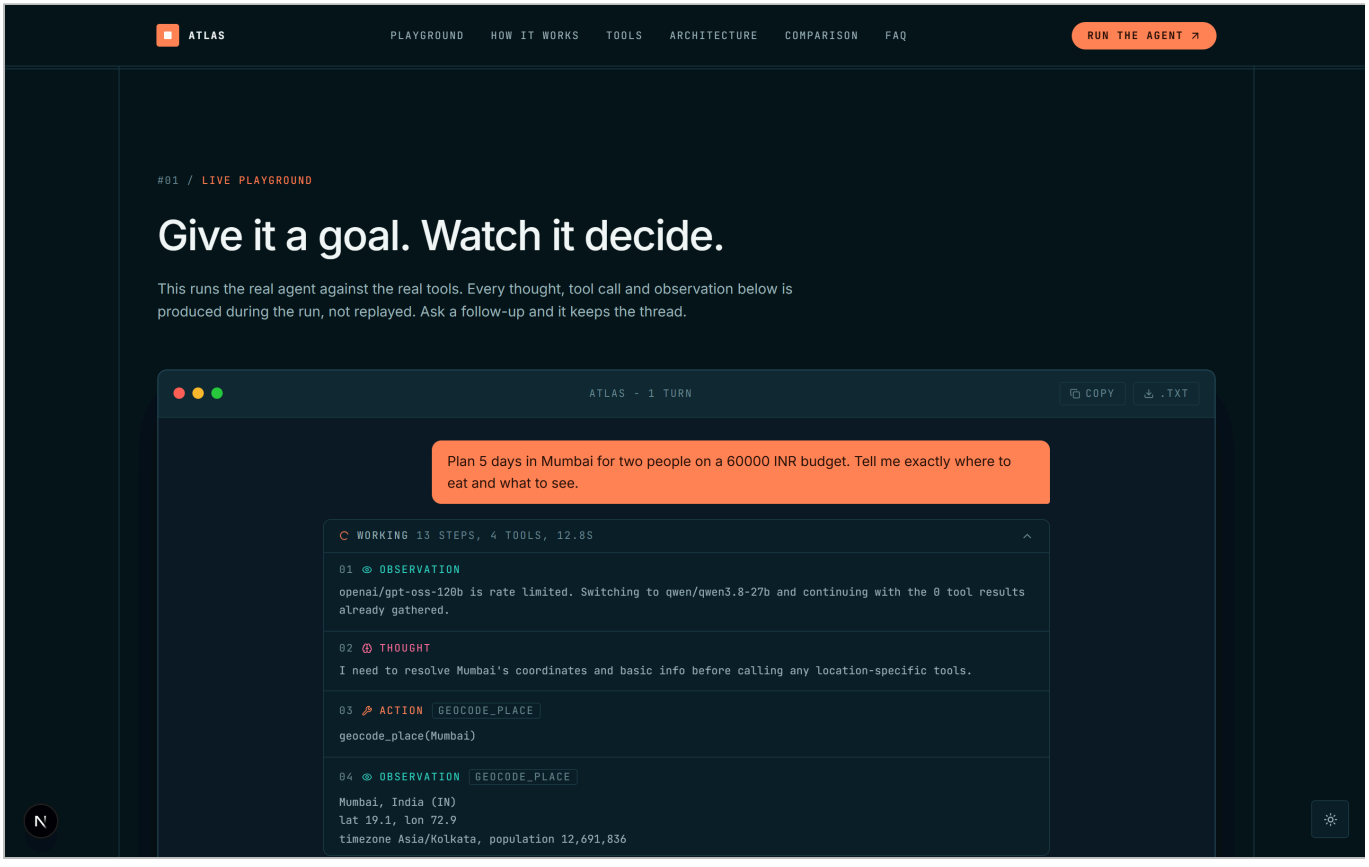


Figure 2: the reasoning trace streaming mid-run. Thought, action and observation steps appear as they complete.

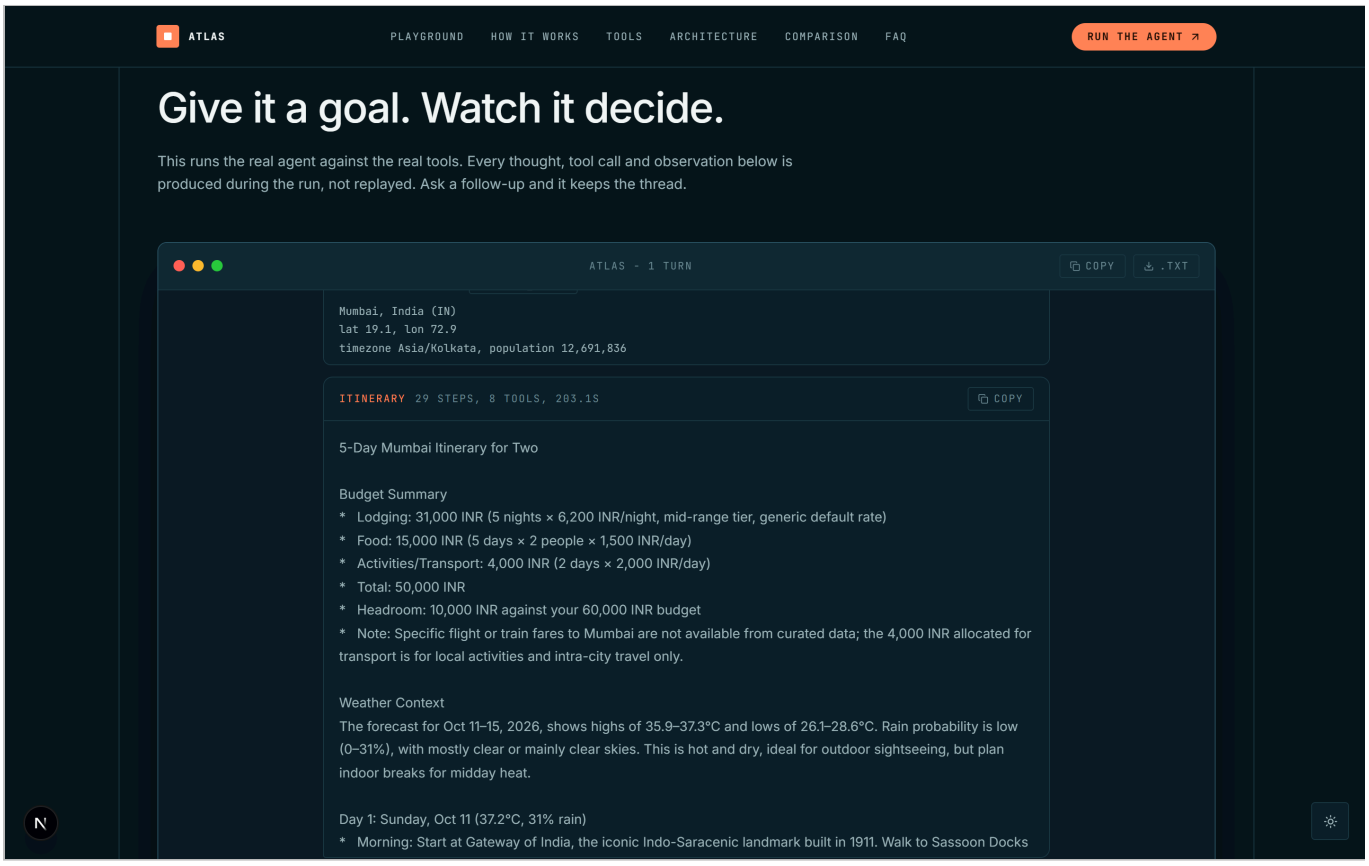


Figure 3: the completed itinerary. The header reports 29 steps, 8 tools and 203.1 s. The plan names the Gateway of India and Sassoon Docks, and states explicitly where fare data was unavailable.

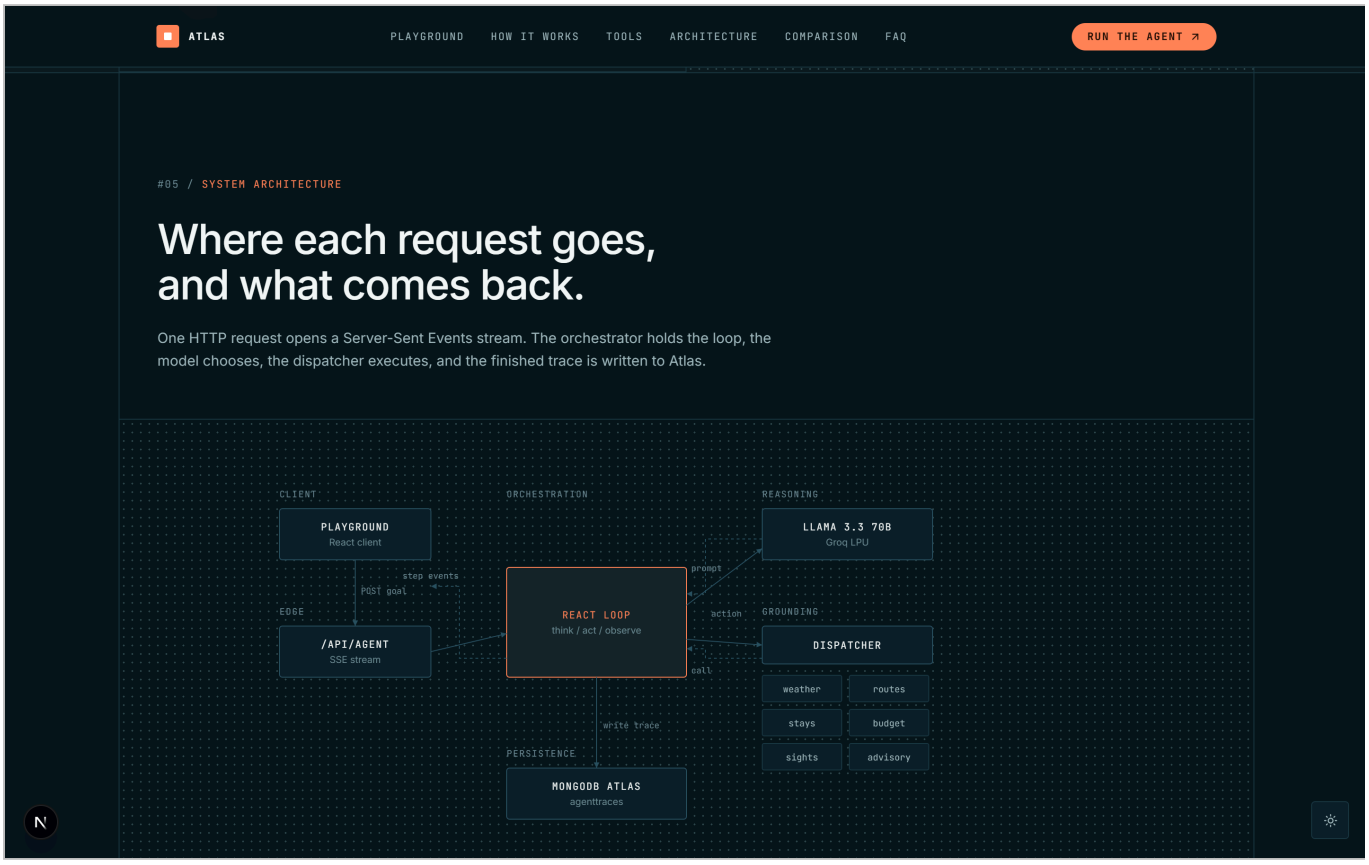


Figure 4: the rendered architecture diagram in the interface.

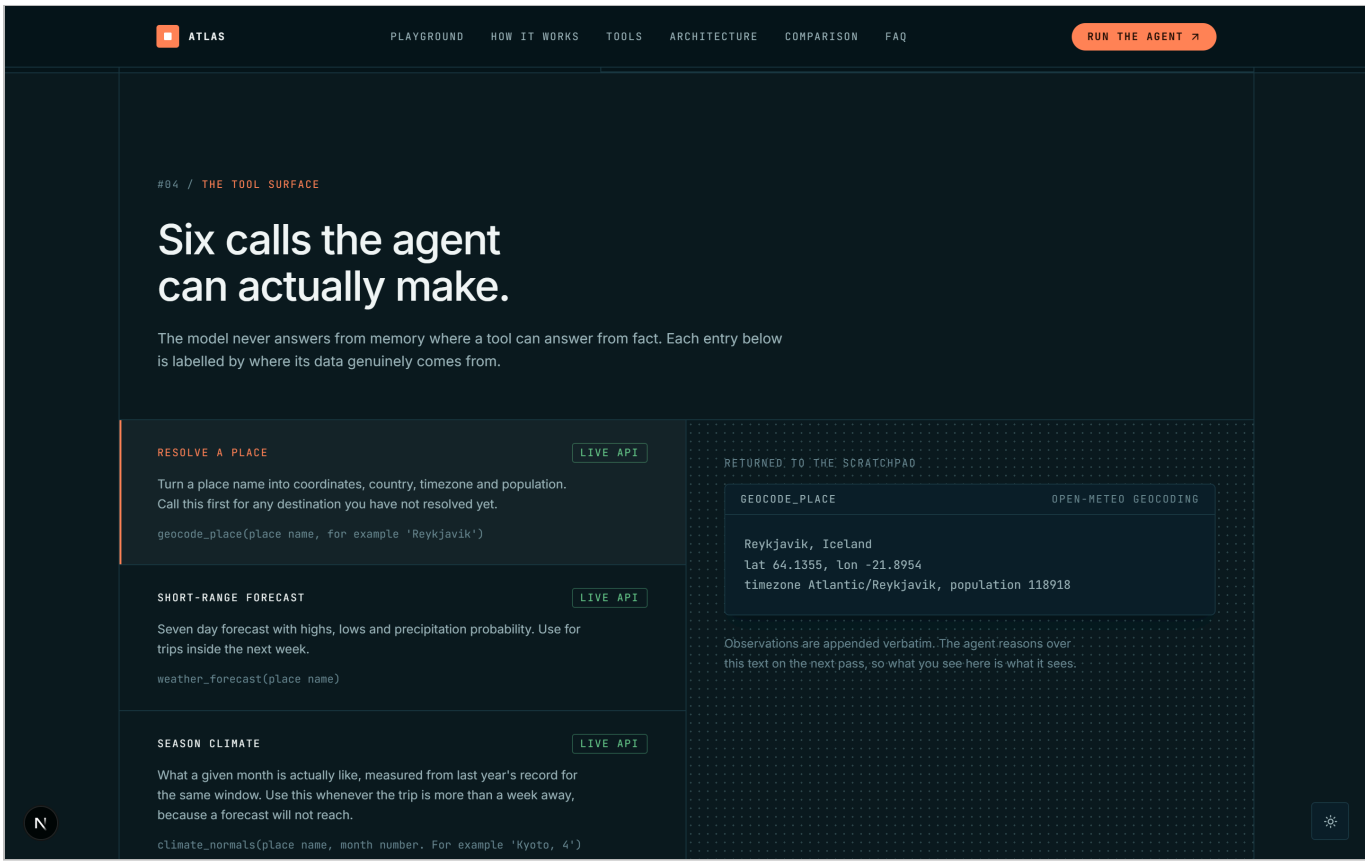


Figure 5: the tool surface, with every tool labelled live, local compute or curated table.

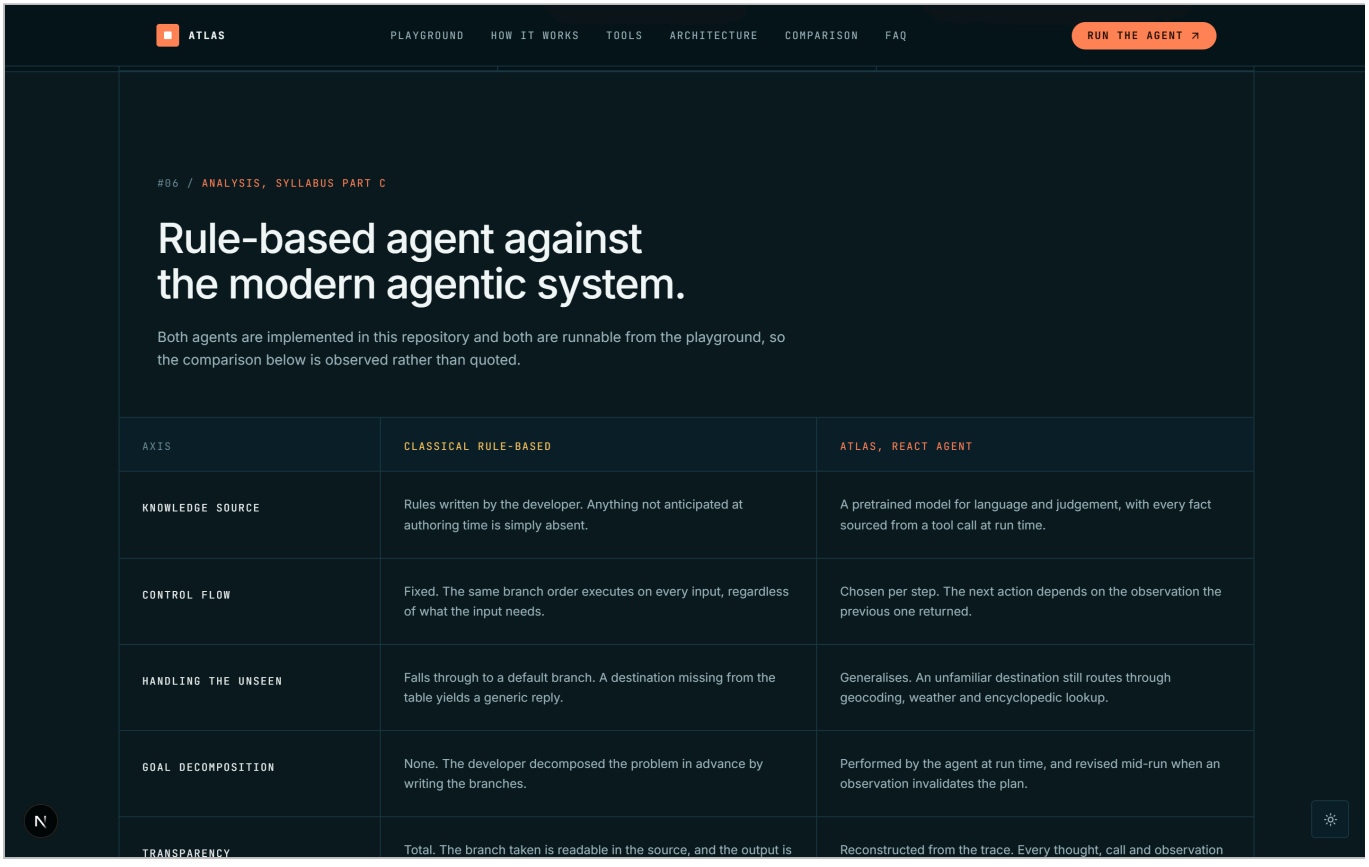


Figure 6: the Part C comparison matrix in the interface.

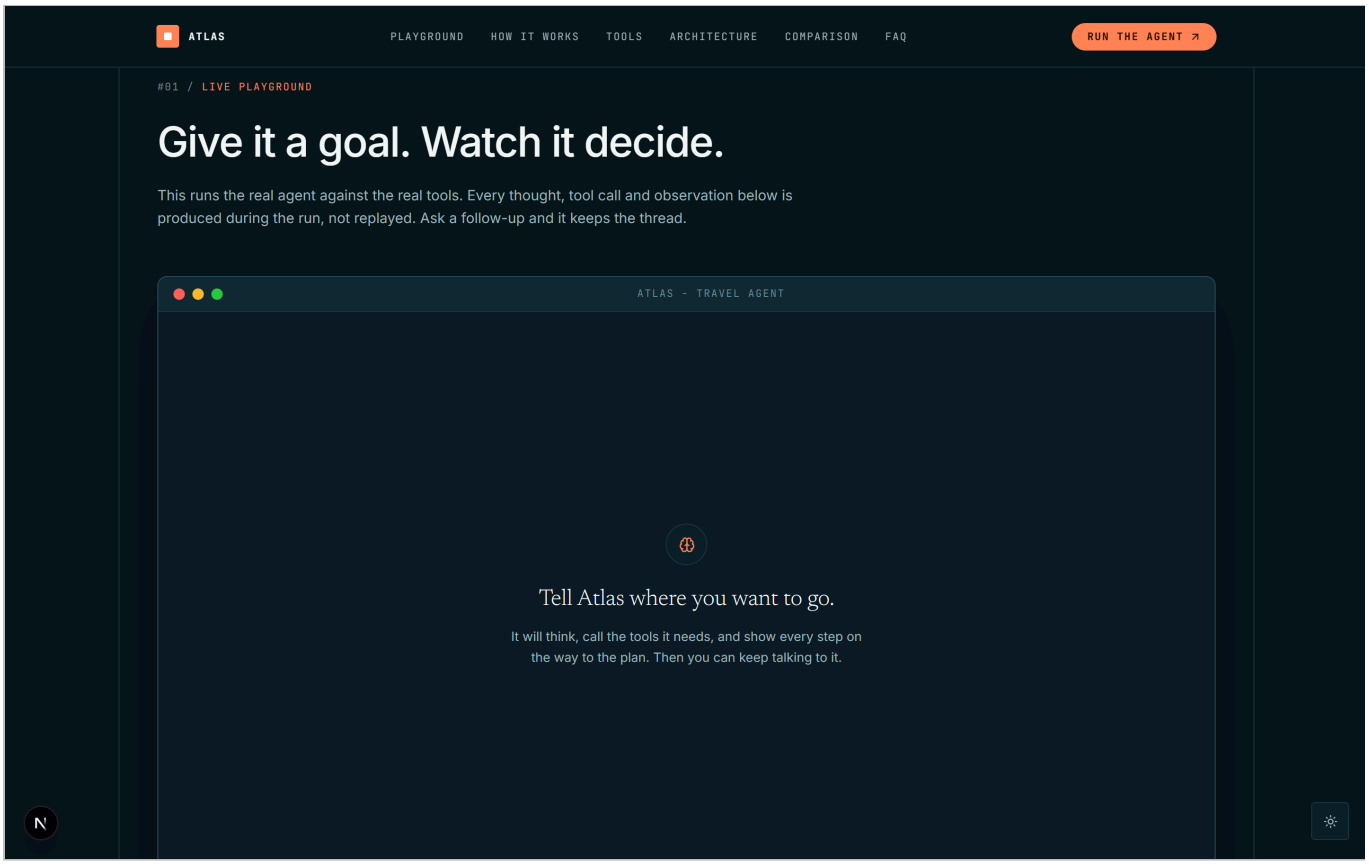


Figure 7: the landing page.

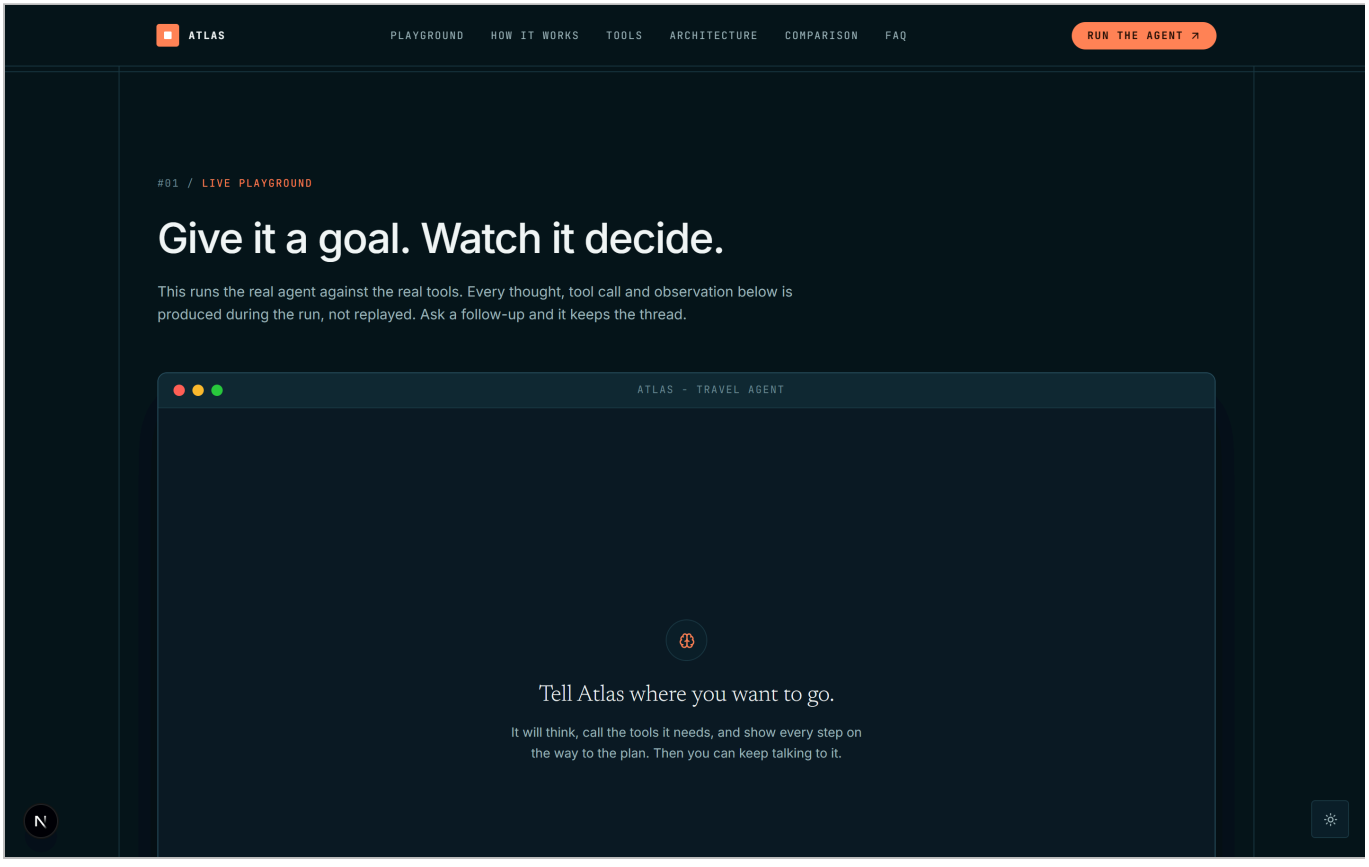


Figure 8: the console before a run, with suggested goals.

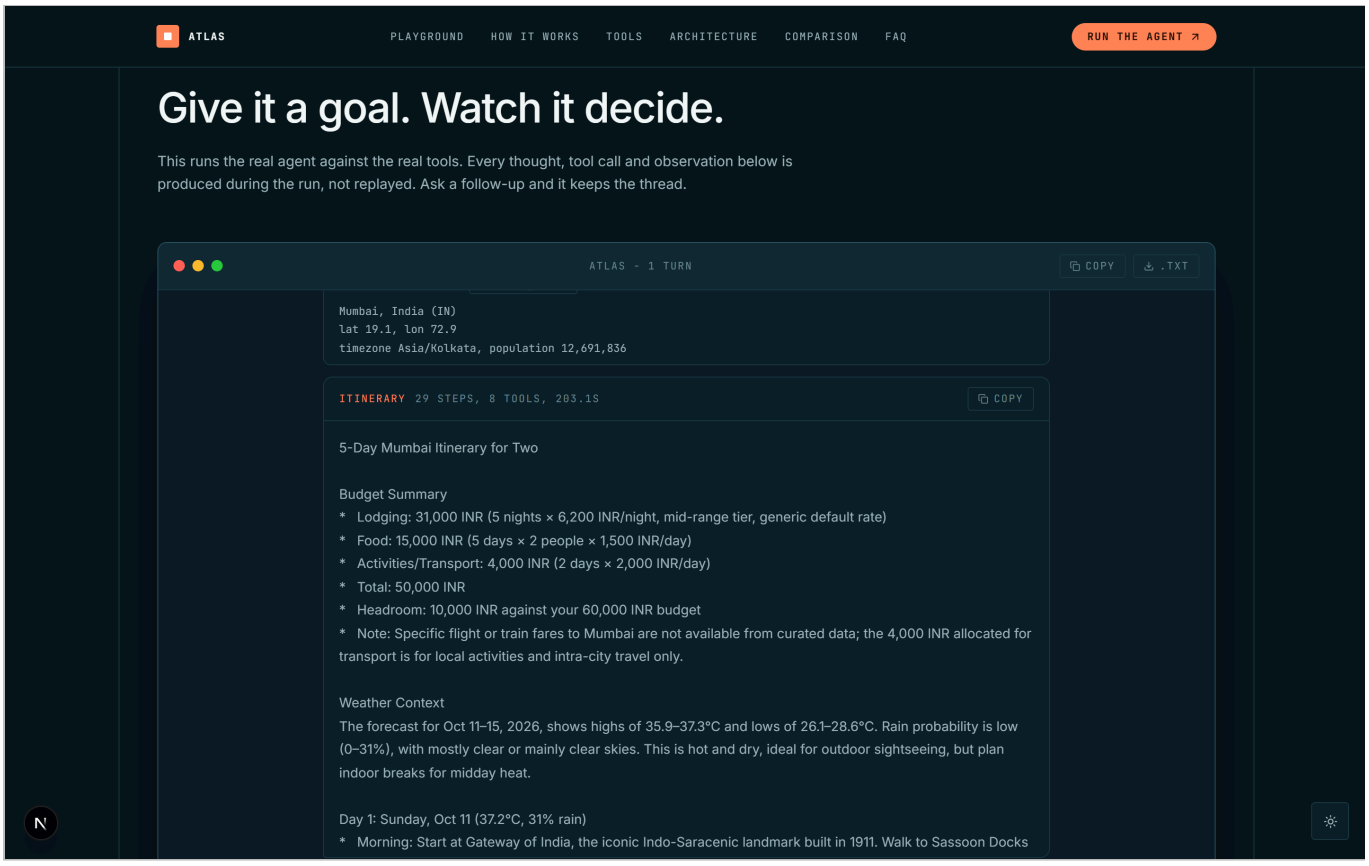


Figure 9: a completed run. The reasoning disclosure reports the step count, the number of distinct tools called and the elapsed time, and can be collapsed once the plan is read.

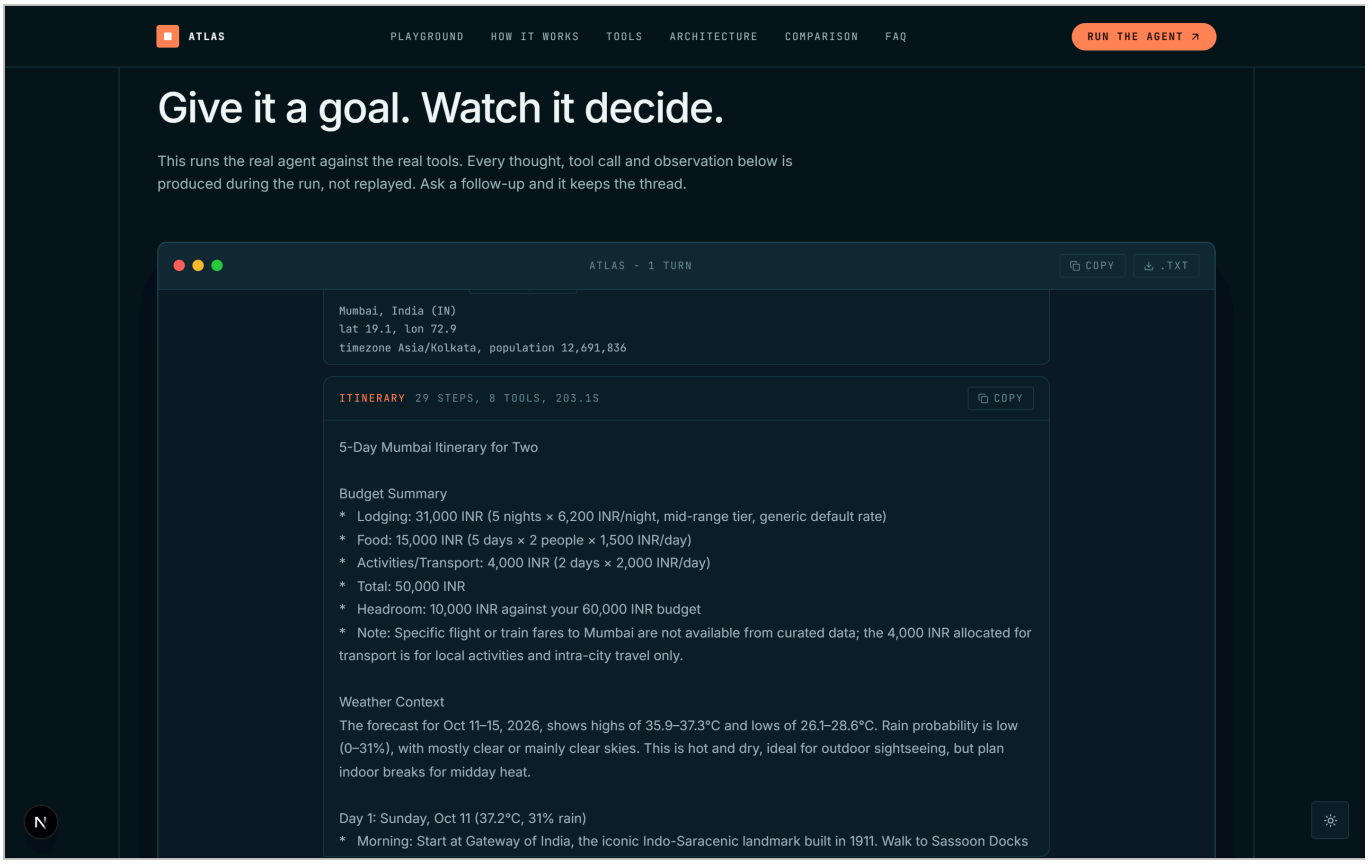


Figure 10: the five phases of one pass of the ReAct cycle, as presented in the interface.

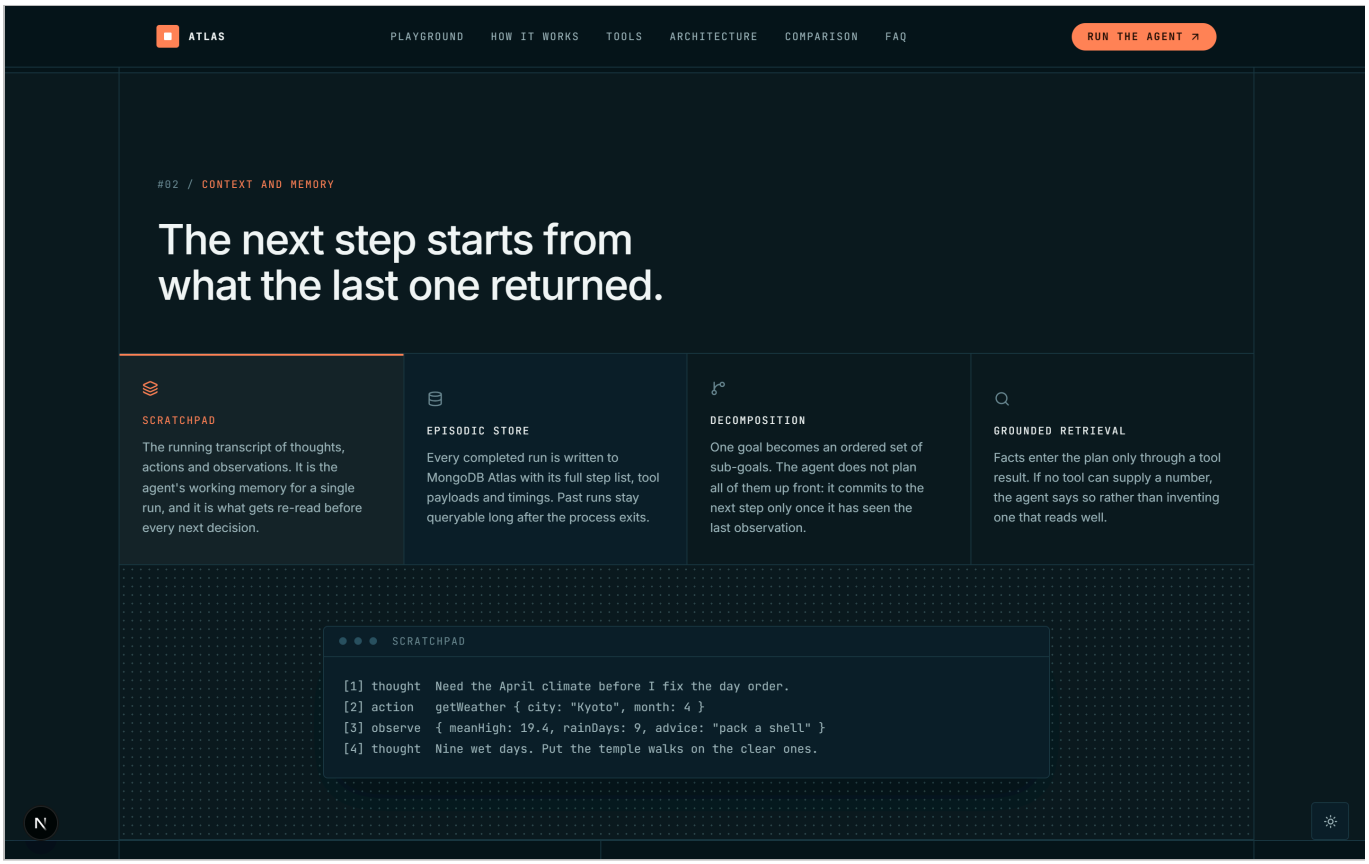


Figure 11: the memory model, showing the scratchpad, the episodic store, decomposition and grounded retrieval.

Representative output

Extract from the live run shown in Figure 3:

```
5-Day Mumbai Itinerary for Two

Budget Summary
*  Lodging: 31,000 INR (5 nights x 6,200 INR/night, mid-range tier)
*  Food: 15,000 INR (5 days x 2 people x 1,500 INR/day)
*  Activities/Transport: 4,000 INR (2 days x 2,000 INR/day)
*  Total: 50,000 INR
*  Headroom: 10,000 INR against your 60,000 INR budget
*  Note: Specific flight or train fares to Mumbai are not available from
   curated data; the 4,000 INR allocated for transport is for local
   activities and intra-city travel only.

Weather Context
The forecast for Oct 11-15, 2026, shows highs of 35.9-37.3 C and lows of
26.1-28.6 C. Rain probability is low (0-31%), with mostly clear skies. This
is hot and dry, ideal for outdoor sightseeing, but plan indoor breaks for
midday heat.

Day 1: Sunday, Oct 11 (37.2 C, 31% rain)
*  Morning: Start at Gateway of India, the iconic Indo-Saracenic landmark
   built in 1911. Walk to Sassoon Docks...
```

Three things are worth noting: the budget line items trace to `lodging_tiers` and `budget_calculator`; the weather paragraph traces to `weather_forecast`; and the agent explicitly states where data was unavailable instead of inventing a fare.

Discussion, takeaway

The gap between a demo and a system is error handling. The ReAct loop itself took an afternoon. Everything after that was failure modes: a model id the key could not serve, a parser that broke on bold text, rate limits, an agent that wrote budgets it had not computed, and map data that returned municipal parks when asked for landmarks. The loop is the easy part, and we would not have known that without running it live.

Prompt instructions are requests, not constraints. We told the model, in capitals, never to state a figure it had not obtained from a tool. It did so anyway. Every reliability property the system actually has is enforced in the orchestrator: the gate, the caps, the budgeting, the deterministic fallback. The transferable lesson is that **if a property matters, it belongs in code around the model, not in the prompt.**

Tool output is part of the prompt. Our tools initially returned raw JSON and the agent reasoned poorly over it. Rewriting every tool to return short labelled lines plus an interpretation sentence ("Nine wet days. Build the plan so outdoor days can move.") improved step quality more than any change to the system prompt. A tool is not just a data source; it is where you tell the agent what the data means.

Data source quality dominates model quality. Switching the place tools from OpenStreetMap to Wikivoyage changed the output from unusable to genuinely good, with no change to the agent at all. We spent far longer tuning the model prompt than choosing the data source, and the data source mattered more.

Honesty is a feature, not a cost. Labelling three tools as curated rather than implying they are live costs nothing and makes the whole system defensible. The agent saying "the international fare alone exceeds your ceiling" is more useful than a tidy plan that quietly ignores it.

Degradation should be visible. When the model is rate limited, the trace says so, says what it is doing about it, and says how many tool results are being preserved. An agent that explains its own failure is far easier to trust than one that silently produces a worse answer.

Limitations we are aware of. Three tools are curated tables rather than live APIs. Free-tier rate limits can interrupt a long run. Wikivoyage coverage is uneven for small towns. We compared behaviour on a handful of prompts rather than a scored benchmark. The system is English only and uses a single agent with no delegation. The next things we would build are a scored evaluation harness that automatically checks every figure traces to an observation, parallel tool calls for the independent lookups, and a reflection step in which the agent critiques its own draft against the goal before returning it.

Part C - Analysis & Comparison

Both agents are implemented in this repository and both run from the same console, in `react`, `classical` or side-by-side `compare` mode. The table below therefore reports observed behaviour rather than textbook claims.

The classical agent (`src/lib/classicalAgent.ts`, 177 lines) is a genuine rule-based agent, not a strawman. It implements four condition-action rules over a destination table:

Rule	Function
Rule 1	Destination pattern matcher, with an explicit fallback branch
Rule 2	Static itinerary lookup table
Rule 3	Budget keyword evaluator, three fixed bands, no arithmetic
Rule 4	Output generator from a fixed template

It is instrumented to report the limitations it encounters at run time:

```
"Unknown destination: Classical agent cannot infer context or extract
 unseen city names."
"Weather constraint ignored: Classical agent has no live API tool integration."
"Flight specifics ignored: Static system cannot query airline schedules."
"Currency calculation ignored: No dynamic mathematical evaluator."
```

Comparison matrix

Axis	Classical Agent (Syllabus)	Modern Agent (This Activity)
Basis	Rule-based	LLM-driven
Logic	Static logic	Dynamic planning
Memory	No memory	Contextual memory
Reasoning	Single-step	Multi-step reasoning
Knowledge source	Rules written by the developer. Anything not anticipated at authoring time is absent.	Pretrained model for language and judgement, with every fact sourced from a tool call at run time.
Control flow	Static. The same branch order executes on every input.	Dynamic. The next action depends on the observation the previous one returned.
Handling the unseen	Falls through to a default branch. A destination missing from the table yields a generic reply.	Generalises. An unfamiliar destination still routes through geocoding, climate, daylight and guide lookup.
Goal decomposition	None. The developer decomposed the problem in advance by writing the branches.	Performed at run time, and revised mid-run when an observation invalidates the plan.
Memory detail	None. Each query is independent.	Scratchpad within a run, contextual across turns, episodic across runs in MongoDB.
Reasoning depth	Single pass, 4 rules.	Multi-step, 9 to 16 iterations observed.
Tool use	None.	16 tools, 11 live.
Transparency	Total. The branch taken is readable in the source and output is reproducible to the character.	Reconstructed from the trace. Every step is logged, but runs are not bit-identical.
Failure mode	Silent irrelevance. Confidently returns a rule that does not fit.	Wasted steps, or fabrication if ungoverned. Mitigated by the four mechanisms in Part B.
Latency	Under 2 ms.	61 s to 240 s.
Cost	Zero.	Token cost per run, free tier here.
Maintenance	Every new city, season or currency is a new branch. Effort grows linearly with coverage.	Add a tool and describe it. Capability grows without rewriting control flow.

Observed comparison on the same prompt

Prompt: *"A week in Reykjavik in November. I have not set a budget."*

Classical agent. Evaluated its destination rules, matched none because Reykjavik is not in its table, fell through to the default branch and returned a generic template in **under 2 ms**. It reported the limitation honestly because we instrumented it to, but it had nothing to say about the destination.

```
Rule 1: Destination Pattern Matcher     -> no match
Rule 1 Fallback                         -> matched
Limitation: Unknown destination. Classical agent cannot infer context
            or extract unseen city names.
```

Atlas. Geocoded Reykjavik, pulled the November climate record, and called `daylight_hours`, which returned:

```
Reykjavik on 2026-11-15
sunrise 09:53, sunset 16:31 (Atlantic/Reykjavik)
daylight 6h 38m
Very short day. Plan one outdoor block and keep it early.
```

That observation changed the plan: the agent allocated one outdoor block per day instead of two and said why. **No rule in the classical agent encodes "at high latitude in winter, daylight is the binding constraint", and no rule could, without somebody first anticipating that specific case.**

A second observed difference, from the Kyoto runs: when the first `budget_calculator` call returned a total over the stated ceiling, the agent issued a second call at the backpacker tier and reported the swap. The classical agent has no mechanism to react to the result of its own computation, because it does not compute anything.

Where the classical agent is genuinely better

This deserves stating plainly, because the comparison is not one-sided and a report that pretended otherwise would be dishonest.

When the domain is actually closed, rules win. The rule agent answers in microseconds, costs nothing, never fabricates, requires no network, and can be proved correct by reading the source. Its output is reproducible to the character, which matters for anything auditable. For a fixed list of twelve destinations with fixed packages, **Atlas is strictly worse engineering**: slower by five orders of magnitude, more expensive, dependent on third-party availability, and capable of being wrong in ways a branch cannot be.

The argument for the agentic approach is that travel planning is **not** a closed domain. The thirteenth city, an unseasonal forecast, a budget that does not divide cleanly, a traveller who asks a follow-up: each is a branch nobody wrote. Atlas absorbs these because it decides the next step after seeing the last result. That is the capability being demonstrated, and it is precisely the one the rule-based formalism cannot express.

The honest summary is that these are not competitors. Rules are the right tool for closed, auditable, latency-critical problems. Agents are the right tool for open-ended problems where the steps cannot be enumerated in advance.

Atlas: An Autonomous Travel Planning Agent - Conclusion

Atlas plans multi-day trips by reasoning in a loop rather than generating in one pass. It implements the ReAct pattern over a 120-billion-parameter model on Groq, dispatches 16 tools of which 11 make live third-party calls, maintains memory at three levels, and streams every thought, action and observation so its behaviour can be audited rather than trusted.

The project met every requirement of the brief:

Required	Delivered
One LLM-based agent	`openai/gpt-oss-120b` on Groq, ReAct loop in `src/lib/agent.ts`
At least two tools	16 tools, 11 of them live third-party APIs
Multi-step reasoning	9 to 16 loop iterations per run, observation-driven
Observable agent behaviour	SSE-streamed traces, four committed logs, MongoDB persistence
Comparison with a classical agent	Implemented in the same repository, runnable side by side
PDF report	This document
Architecture and tool flow diagrams	Part B
Code repository with working demo	github.com/Aryan-lomte05/Atlas, plus `demo_agent.py`
Agent interaction logs	`docs/logs/`, four runs
Prompts and output screenshots	`docs/screenshots/`, eleven images

The result we found most interesting was not that the agentic approach beat the rule-based one, which the brief rather presupposes. It was **how much of the reliability had to be enforced outside the model.** The gate that refuses an ungrounded answer, the caps that stop the agent exploring forever, the budgeting that reserves steps for what matters, and the deterministic fallback that computes the total when the model will not, are all orchestration code.

The model supplies fluency and judgement. Everything you can actually depend on is built around it.

References

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. arXiv:2210.03629
2. Russell, S. and Norvig, P. *Artificial Intelligence: A Modern Approach*. Chapters 2 and 3, on intelligent agents, environments and problem-solving agents.
3. Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022. arXiv:2201.11903
4. Schick, T. et al. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. arXiv:2302.04761
5. Groq API documentation. https://console.groq.com/docs
6. Open-Meteo API documentation. https://open-meteo.com/en/docs
7. Wikivoyage, the free worldwide travel guide. https://en.wikivoyage.org (CC BY-SA)

8. OpenStreetMap Overpass API. https://wiki.openstreetmap.org/wiki/Overpass_API (ODbL)
9. Frankfurter exchange rate API, built on ECB reference rates. https://www.frankfurter.app
10. Nager.Date public holiday API. https://date.nager.at
11. Wikipedia REST API. https://en.wikipedia.org/api/rest_v1/
12. sunrise-sunset.org API. https://sunrise-sunset.org/api
13. Next.js documentation. https://nextjs.org/docs
14. mathjs documentation. https://mathjs.org/docs
15. Photography from Unsplash under the Unsplash License, credited in `public/img/CREDITS.md`